

Komposition

Thema	Theorie	Technische Details
Definition	Komposition ist eine starke Form der Assoziation, bei der ein Objekt (Ganzes) ein anderes Objekt (Teil) besitzt. Das Teil-Objekt kann nicht unabhängig existieren und wird zusammen mit dem Ganz-Objekt erzeugt und zerstört.	Das Teil-Objekt wird innerhalb der Hauptklasse instanziiert und ist nicht separat verwendbar. Die Lebensdauer des Teil-Objekts ist an das Ganz-Objekt gebunden. Keine Setter für das Teil-Objekt, um externe Manipulationen zu verhindern. Direkte Speicherung des Teil-Objekts als <code>private final</code> Attribut, um Unveränderlichkeit sicherzustellen.
Unterschiede zur Aggregation	Aggregation ist eine schwächere Beziehung als Komposition, bei der das Teil-Objekt unabhängig existieren kann und von mehreren Ganz-Objekten referenziert werden kann.	Aggregation erlaubt eine unabhängige Existenz des Teil-Objekts, während Komposition dies verhindert. Aggregation erlaubt Setzen und Entfernen des Teil-Objekts, während Komposition es intern verwaltet. Bei Aggregation kann das Teil-Objekt auch von anderen Klassen referenziert und manipuliert werden.
Konstruktion des Teil-Objekts	Das Teil-Objekt wird direkt im Konstruktor des Ganz-Objekts oder als feste Instanzvariable erstellt.	Das Teil-Objekt wird entweder als <code>private final</code> Attribut oder durch Konstruktorinitialisierung gesetzt. Es gibt keine öffentliche Methode, um das Teil-Objekt von außen zu überschreiben. Falls eine flexiblere Initialisierung erforderlich ist, kann ein Factory-Pattern oder ein Builder-Pattern verwendet werden.
Lebenszyklus des Teil-Objekts	Das Teil-Objekt existiert nur solange das Ganz-Objekt existiert und wird mit diesem zerstört.	Wird das Ganz-Objekt gelöscht, wird auch das Teil-Objekt entfernt (keine unabhängige Referenzierung möglich). Kein expliziter Garbage Collection-Mechanismus notwendig, da die Komposition automatisch sicherstellt, dass keine externen Referenzen bestehen. Falls notwendig, kann <code>finalize()</code> oder <code>AutoCloseable</code> implementiert werden, um Ressourcen explizit zu verwalten.

Thema	Theorie	Technische Details
Zugriffsbeschränkungen und Kapselung	Das Teil-Objekt wird vor direktem Zugriff geschützt und nur über die Methoden der Hauptklasse verwendet.	Direkter Zugriff von außen wird durch <code>private</code> oder <code>protected</code> Attribute verhindert. Falls das Teil-Objekt Funktionen bereitstellen muss, sollten Methoden der Hauptklasse als Vermittler fungieren. Vererbung oder Interfaces können genutzt werden, um Abhängigkeiten weiter zu minimieren.
Speicherverwaltung und Garbage Collection	Durch die starke Bindung zwischen Ganz und Teil-Objekt gibt es keine ungewollten Speicherlecks.	Da das Teil-Objekt ohne das Ganz-Objekt nicht existiert, gibt es keine ungenutzten Objekte im Speicher. Die JVM kann das gesamte Objekt (Ganzes + Teil) bei Nichtverwendung effizient aufräumen. Falls das Teil-Objekt externe Ressourcen verwaltet (z. B. Dateizugriffe oder Netzwerkverbindungen), sollte sichergestellt werden, dass diese ordnungsgemäß freigegeben werden.
Best Practices und Design Patterns	Die Komposition kann mit weiteren Design Patterns kombiniert werden, um Flexibilität und Wartbarkeit zu verbessern.	Builder-Pattern: Ermöglicht die schrittweise Konstruktion eines komplexen Objekts mit mehreren Teil-Objekten. Factory-Pattern: Verwaltet die Erstellung des Teil-Objekts in einer separaten Klasse, um die Hauptklasse zu entkoppeln. Composite-Pattern: Falls mehrere Teil-Objekte hierarchisch organisiert werden müssen, kann eine rekursive Kompositionsstruktur verwendet werden. Dependency Injection: Falls das Teil-Objekt konfigurierbar sein soll, kann es durch Abhängigkeitsinjektion bereitgestellt werden, allerdings bleibt es in der Hauptklasse unveränderlich.
Testbarkeit und Robustheit	Die Komposition kann durch Unit-Tests abgesichert werden, um Fehler frühzeitig zu erkennen.	Unit-Tests prüfen, ob das Teil-Objekt korrekt erstellt und verwaltet wird. Mocking-Frameworks wie Mockito helfen, komplexe Abhängigkeiten im Testumfeld zu simulieren. Code-Analyse-Tools können genutzt werden, um zyklische Abhängigkeiten oder fehlerhafte Speicherfreigaben zu erkennen. Edge Cases sollten berücksichtigt werden, z. B. ungültige oder nicht initialisierte Teil-Objekte.